

DSA 期末错题总结：按对话代码来源校订版
题干样例、错误梳理、对话来源代码与考前模板

依据你上传的《题库.docx》和项目对话整理

2026 年 6 月

目录

使用说明	2
1 树、二叉树、BST 与堆	2
1.1 22: 根据二叉树前中序序列建树	2
1.2 23: 括号嵌套二叉树	3
1.3 24: 二叉树	5
1.4 25: 二叉搜索树的层次遍历	6
1.5 26: 实现堆结构	7
1.6 27: 树的转换	9
1.7 28: Huffman 编码树	10
1.8 49: 二叉树的深度	12
1.9 50: 扩展二叉树	13
1.10 53: 根据二叉树中后序序列建树	14
1.11 54: 验证二叉搜索树	16
2 图、BFS、最短路、生成树与拓扑	17
2.1 1: 单词序列	17
2.2 2: 仙岛求药	20
2.3 3: 变换的迷宫	22
2.4 1: 拓扑排序	24
2.5 2: 丛林中的路	26
2.6 3: A Walk Through the Forest	28
2.7 4: 打怪救公主	31
2.8 5: 兔子与樱花	33
2.9 56: 拯救行动	36
2.10 57: 连接所有点的最小费用	38
3 全局输入与调试清单	41
4 考前模板索引	41

使用说明

这份最终版严格沿用你上传的题库 Word 文档中的原始题号与题名。例如，树题使用 **22: 根据二叉树前中序序列建树**，图题中也保留题库里的重复编号，例如 **1: 单词序列**和 **1: 拓扑排序**。章节结构仍按复习逻辑分为“树、二叉树、BST 与堆”和“图、BFS、最短路、生成树与拓扑”。每道题不再堆放旧错误代码，而是把你多轮修改中暴露出来的问题合并成简明复盘点。本版另外校订了“最终代码”的来源：不再把本次重新写出的代码当作最终代码；所有代码均按对话来源保留，能确认是你贴出的版本则优先采用，不能确认用户最终版的题目采用此前对话中已经给出的可用代码，并在代码前统一说明。

1 树、二叉树、BST 与堆

1.1 22: 根据二叉树前中序序列建树

题干与样例

题目编号与名称：22: 根据二叉树前中序序列建树

要求：已知二叉树的前序遍历和中序遍历，结点为互不相同的大写字母，输出后序遍历。多组数据，每组两行。

输入：多组数据；每组第一行是前序序列，第二行是中序序列，长度不超过 26。

输出：对每组数据输出一行后序遍历序列。

样例输入：

```
DURPA
RUDPA
XTCNB
CTBNX
```

样例输出：

```
RUAPD
CBNTX
```

错误梳理

- 不要真的先建完整结点对象也可以；本题只要求输出后序，递归字符串切片最直接。
- 前序第一个字符一定是根；在中序中找到根，左边是左子树，右边是右子树。
- 切分前序时要用左子树长度，不能按字符位置随便截。
- 这是多组数据，不能只读两行；要循环读到 EOF。
- 递归出口是空串返回空串，单结点也会自然返回根。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 1: 最终可用代码（对话来源）

```
class Treenode:
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
        self.right = right

def Buildtree(preorder, inorder):
    if not preorder:
        return None
```

```

root_val = preorder[0]
root = Treenode(root_val)

x = inorder.index(root_val)

# 中序遍历: 左子树 根 右子树
ino_left = inorder[:x]
ino_right = inorder[x + 1:]

# 左子树结点个数
l = len(ino_left)

# 前序遍历: 根 左子树 右子树
pre_left = preorder[1:1 + l]
pre_right = preorder[1 + l:]

root.left = Buildtree(pre_left, ino_left)
root.right = Buildtree(pre_right, ino_right)

return root

def postorder(root, ans):
    if root:
        postorder(root.left, ans)
        postorder(root.right, ans)
        ans.append(root.val)

while True:
    try:
        line1 = input()
        line2 = input()
        root = Buildtree(line1, line2)
        ans = []
        postorder(root, ans)
        print(''.join(ans))
    except EOFError:
        break

```

复盘记忆点

- 前序 + 中序：根在前序开头，靠中序划分左右子树。
- 输出后序时顺序是 left + right + root。
- 多组到 EOF 的题，用 sys.stdin 读完整输入更稳。

1.2 23: 括号嵌套二叉树

题干与样例

题目编号与名称：23: 括号嵌套二叉树

要求：给出括号嵌套表示的二叉树，输出前序遍历和中序遍历。星号 * 表示空树；形如 A(B(*,C),D(E))。

输入：第一行 n；接下来 n 行，每行一棵树的括号嵌套表示。

输出：对每棵树输出两行：前序遍历和中序遍历。

样例输入：

```

2
A
A(B(*,C),D(E))

```

样例输出：

```

A
A
ABCDE
BCAED

```

错误梳理

- 这题最容易错在解析括号：不能简单按逗号 split，因为逗号可能出现在更深层括号里。
- 根一定是当前子串第一个字符；如果后面没有括号，就是叶子。
- 左子树为空、右子树非空时写作 A(*,R)，不能把 * 当普通结点输出。
- 题面文字提到后序，但输出要求只有前序和中序，要以输出要求为准。
- 多棵树逐行处理，不要把全部输入拼成一个大字符串。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 2: 最终可用代码（对话来源）

```
class Treenode:
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
        self.right = right
def buildtree(s,i):
    if s[i] == "*":
        return None, i+1
    root = Treenode(s[i])
    i += 1
    if i < len(s) and s[i] == "(":
        i+=1
        root.left, i = buildtree(s, i)
    if i < len(s) and s[i] == ",":
        i += 1
        root.right, i = buildtree(s,i)

    if i < len(s) and s[i] == ")":
        i+=1

    return root,i

def preorder(root, ans):
    if root:
        ans.append(root.val)
        preorder(root.left, ans)
        preorder(root.right, ans)

def inorder(root, ans):
    if root:
        inorder(root.left, ans)
        ans.append(root.val)
        inorder(root.right, ans)

n = int(input())
for i in range(n):
    s = input()
    root, j= buildtree(s,0)
    ans1 = []
    ans2 = []
    preorder(root, ans1)
    inorder(root, ans2)
    print("".join(ans1))
    print("".join(ans2))
```

复盘记忆点

- 括号解析题的关键是“只切最外层逗号”。
- 遇到 * 返回 None，遍历时 None 输出空串。
- 输出什么遍历要看“输出”部分，不要被描述里多余的话带偏。

1.3 24: 二叉树

题干与样例

题目编号与名称：24: 二叉树

要求：完全二叉树编号为 $1, 2, 3, \dots, n$ ，求编号 m 所在子树中实际存在的结点个数。

输入：多组数据，每行两个整数 m, n ；0 0 结束。

输出：每组输出一行，表示以 m 为根的子树中编号不超过 n 的结点个数。

样例输入：

```
3 12
0 0
```

样例输出：

```
4
```

错误梳理

- 不要建树： n 最大到 $1e9$ ，建结点一定不合适。
- 完全二叉树中， m 的下一层不是一个结点，而是连续区间 $[2m, 2m+1]$ 。
- 每一层都要与 n 截断，右端超过 n 时只能统计到 n 。
- 输入是多组，以 0 0 结束；你之前容易只处理一组。
- 题面输入顺序是 m, n ，不要写反成 n, m 。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 3: 最终可用代码（对话来源）

```
while True:
    n, m = map(int, input().split())
    if n==0 and m==0:
        break
    ans = 0
    right = n
    left = n
    while left <= m:
        ans += min(m, right) - left + 1
        right = right*2 + 1
        left = left*2
    print(ans)
```

复盘记忆点

- 堆式编号题优先想“每层区间扩张”。
- while 条件看 left 是否仍然存在： $left \leq n$ 。

- 贡献公式： $\min(\text{right}, n) - \text{left} + 1$ 。

1.4 25: 二叉搜索树的层次遍历

题干与样例

题目编号与名称：25: 二叉搜索树的层次遍历

要求：输入一行无序整数，重复数字只插入一次，建立二叉搜索树后输出层次遍历。

输入：一行若干整数，空格分隔。

输出：一行输出 BST 的层次遍历结果，数字之间一个空格，末尾无多余空格。

样例输入：

```
51 45 59 86 45 4 15 76 60 20 61 77 62 30 2 37 13 82 19 74 2 79 79 97 33 90 11 7 29 14 50 1 96 59 91 39 34 6 72 7
```

样例输出：

```
51 45 59 4 50 86 2 15 76 97 1 13 20 60 77 90 11 14 19 30 61 82 96 7 29 37 62 79 91 6 33 39 74 34 72
```

错误梳理

- 重复值只计入一次，不能简单把相等元素插到左边或右边。
- 层次遍历要用队列；用 `list.pop(0)` 可以过小数据，但 `deque` 更规范。
- 输出末尾不能多空格，最好先收集成字符串列表再 `join`。
- 插入函数中 `key == node.key` 时直接返回，不能继续递归。
- 如果输入为空要避免访问 `root`，不过本题通常保证有数字。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 4: 最终可用代码（对话来源）

```
class TreeNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None
    def put(self, key):
        if self.root:
            self._put(key, self.root)
        else:
            self.root = TreeNode(key)
    def _put(self, key, currentNode):
        if key < currentNode.key:
            if currentNode.left:
                self._put(key, currentNode.left)
            else:
                currentNode.left = TreeNode(key)
        elif key > currentNode.key:
            if currentNode.right:
                self._put(key, currentNode.right)
            else:
                currentNode.right = TreeNode(key)
    def bfs(self, root, ans):
        if not root:
            return
```

```

else:
    queue = [root]
    while queue:
        node = queue.pop(0)
        ans.append(str(node.key))
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
    return ans

lis = list(map(int, input().split()))
hp = BinarySearchTree()
for n in lis:
    hp.put(n)
ans = []
bfs(hp.root, ans)
print(" ".join(ans))

```

复盘记忆点

- BST 插入：小走左，大走右，相等忽略。
- 层次遍历模板：deque + popleft。
- 输出列表用 join，避免末尾空格。

1.5 26: 实现堆结构

题干与样例

题目编号与名称：26: 实现堆结构

要求：维护一个初始为空的数组，支持插入元素和输出删除最小元素，要求用最小堆高效实现。

输入：第一行 t；每组第一行 n；之后 n 行操作。type=1 后接 u 表示插入，type=2 表示输出并删除最小值。

输出：每次 type=2 输出被删除的最小数字。

样例输入：

```

2
5
1 1
1 2
1 3
2
2
4
1 5
1 1
1 7
2

```

样例输出：

```

1
2
1

```

错误梳理

- Python 模块名是 heapq，不是 headq；弹出函数是 heappop，不是 heapop。
- 每组测试数据都要重新建一个空堆，不要把上一组残留带到下一组。
- 操作行长度不固定：type=1 有两个数，type=2 只有一个数。

- 复杂度要求 $O(n \log n)$ ，不能每次删除前 sort。
- 输出多行建议收集到 `ans` 最后统一打印，速度更稳。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 5: 最终可用代码（对话来源）

```
class BinHeap:
    def __init__(self):
        self.heaplist = [0]
        self.size = 0
    def percUp(self, i):
        while i//2 > 0:
            if self.heaplist[i] < self.heaplist[i//2]:
                self.heaplist[i], self.heaplist[i//2] = self.heaplist[i//2], self.heaplist[i]
            i = i//2
    def insert(self, k):
        self.heaplist.append(k)
        self.size += 1
        self.percUp(self.size)
    def FindMin(self, i):
        if i*2 > self.size:
            return i*2
        else:
            if self.heaplist[i*2] < self.heaplist[i*2+1]:
                return i*2
            else:
                return i*2+1
    def percDown(self, i):
        while i*2 <= self.size:
            mc = self.FindMin(i)
            if self.heaplist[i] > self.heaplist[mc]:
                self.heaplist[i], self.heaplist[mc] = self.heaplist[mc], self.heaplist[i]
            i = mc
    def delMin(self):
        val = self.heaplist[1]
        self.heaplist[1] = self.heaplist[self.size] # 注意这里size和实际队列长度的区别
        self.size -= 1
        self.heaplist.pop()
        self.percDown(1) # 这里是要往下沉的所以是1
        return val

t = int(input())

while t:
    hp = BinHeap()
    n = int(input())

    while n:
        op = list(map(int, input().split()))

        if op[0] == 1:
            hp.insert(op[1])
        else:
            print(hp.delMin())

        n -= 1

    t -= 1
```

复盘记忆点

- `heapq` 默认就是最小堆。
- 插入 `heappush`，删除最小 `heappop`。

- 多组测试时，每组 `heap=[]`。

1.6 27: 树的转换

题干与样例

题目编号与名称：27: 树的转换

要求：用 d/u 字符串表示一般树的 DFS 过程，将其按左儿子右兄弟法转换为二叉树，输出转换前和转换后的高度。

输入：一行由 u 和 d 组成的字符串。

输出：按格式 `h1 ==> h2` 输出。

样例输入：

```
dudduduudu
```

样例输出：

```
2 ==> 4
```

错误梳理

- 普通树高度只看向下 d 的最大深度。
- 转换后二叉树高度中，向第一个孩子走和向右兄弟走都会增加边数。
- 你之前容易把“兄弟边”不算进转换后二叉树高度，导致样例对不上。
- 这题可以不真的建树，直接 DFS 模拟时同步维护两个高度更简单。
- 输出格式固定，`==>` 两边有空格。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 6: 最终可用代码（对话来源）

```
class TreeNode:
    def __init__(self, key):
        self.key = key
        self.children = []
        self.parent = None # 普通树: 可以有多个孩子

class BinaryNode:
    def __init__(self, key):
        self.key = key
        self.left = None # 左儿子
        self.right = None # 右兄弟

def convert(root):
    if root is None:
        return None

    # 创建对应的二叉树结点
    b_root = BinaryNode(root.key)

    # 如果没有孩子, 直接返回
    if len(root.children) == 0:
        return b_root

    # 第一个孩子变成左儿子
    b_root.left = convert(root.children[0])
```

```

# 其余孩子依次变成右兄弟
current = b_root.left

for child in root.children[1:]:
    current.right = convert(child)
    current = current.right

return b_root

def height(root):
    if root is None:
        return -1
    else:
        height_left = height(root.left)
        height_right = height(root.right)
        return max(height_left, height_right) + 1

def length(root):
    if root is None:
        return -1
    if len(root.children) == 0:
        return 0
    max_height = 0
    for child in root.children:
        max_height = max(max_height, length(child))
    return max_height + 1

n = input().strip()
count = 0
root = TreeNode(0)
current = root
for i in n:
    if i == 'd':
        count += 1
        nd = TreeNode(count)
        current.children.append(nd)
        nd.parent = current
        current = nd
    elif i == 'u':
        current = current.parent

b_root = convert(root)
print(f"{length(root)} => {height(b_root)}")

```

复盘记忆点

- 原树高度：最大向下层数。
- 左儿子右兄弟二叉树高度：child 和 sibling 边都算。
- 这题不用输出树，只要维护高度。

1.7 28: Huffman 编码树

题干与样例

题目编号与名称：28: Huffman 编码树

要求：给定 n 个外部结点权值，构造 Huffman 树，求最小带权外部路径长度。

输入：第一行 t ；每组第一行 n ，第二行 n 个权值。

输出：每组输出一行最小外部路径长度总和。

样例输入：

```

2
3

```

```
1 2 3
4
1 1 3 5
```

样例输出：

```
9
17
```

错误梳理

- Huffman 的核心不是排序一次，而是每次取当前最小的两个合并。
- 合并产生的新权值要重新放回集合中继续参与比较。
- 答案累加每次合并的和，而不是最后根结点权值。
- 即使 n 小，也建议用堆；手动列表删除容易写错索引。
- 多组数据时每组重新 heapify。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 7: 最终可用代码（对话来源）

```
import heapq

def optimal_binary_tree(weights):
    # 1. 开始 n 个结点位于集合 S
    heapq.heapify(weights)

    total_wpl = 0

    # 2. 每次取出两个权值最小的结点
    while len(weights) > 1:
        n1 = heapq.heappop(weights)
        n2 = heapq.heappop(weights)

        # 构造新结点 r
        r = n1 + n2

        # 这一次合并产生的代价
        total_wpl += r

        # 将 r 加回集合 S
        heapq.heappush(weights, r)

    # 3. total_wpl 就是最小 WPL
    return total_wpl

t = int(input())
for _ in range(t):
    n = int(input())
    weights = list(map(int, input().split())) # 这一行读入 n 个权值
    print(optimal_binary_tree(weights))
```

复盘记忆点

- Huffman 模板：反复取最小两个，合并，答案加合并值。
- heapify 是把普通列表原地变成堆。
- 答案不是编码字符串，而是最小 WPL。

1.8 49: 二叉树的深度

题干与样例

题目编号与名称: 49: 二叉树的深度

要求: 给定编号 1 到 n 的二叉树, 根结点为 1, 求深度。深度按根到叶最长路径上的结点数计算。

输入: 第一行 n; 接下来 n 行, 第 i 行给出结点 i 的左、右孩子编号, -1 表示空。

输出: 输出一个整数, 即二叉树深度。

样例输入:

```
3
2 3
-1 -1
-1 -1
```

样例输出:

```
2
```

错误梳理

- 这题根明确是 1, 不需要再找根。
- 输入行号就是结点编号: 第 i 行描述结点 i, 不能按遍历顺序读。
- 深度按结点数, 不是边数。空树为 0, 叶子为 1。
- n 很小, 但写成通用递归更安全。
- 左右孩子编号为 -1 时不要访问 children[-1]。

代码来源: 本题代码优先沿用你在对话中贴出的最终/可用版本; 如果当前对话记录中没有完整用户最终代码, 则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 8: 最终可用代码 (对话来源)

```
class BinaryNode:
    def __init__(self, key):
        self.key = key
        self.left = None    # 左儿子
        self.right = None   # 右兄弟

def height(root):
    if root is None:
        return 0
    else:
        height_left = height(root.left)
        height_right = height(root.right)
        return max(height_left, height_right) + 1

n = int(input())
nodes = [None] * (n + 1)
for i in range(1, n + 1):
    nodes[i] = BinaryNode(i)
for i in range(1, n + 1):
    a, b = map(int, input().split())
    if a != -1:
        nodes[i].left = nodes[a]
    if b != -1:
        nodes[i].right = nodes[b]
root = nodes[1]

print(height(root))
```

复盘记忆点

- 如果题目写“根结点为 1”，直接 `depth(1)`。
- 数组 `left/right` 比建对象更短。
- 递归出口：编号 -1 深度 0。

1.9 50: 扩展二叉树

题干与样例

题目编号与名称：50: 扩展二叉树

要求：给出扩展二叉树的先序序列，其中 `.` 表示空结点，输出原二叉树的中序和后序。

输入：一行扩展二叉树先序序列，由大写字母和 `.` 组成。

输出：第一行输出中序序列，第二行输出后序序列。

样例输入：

```
ABD..EF..G..C..
```

样例输出：

```
DBFEGAC
DFGEBCA
```

错误梳理

- 扩展先序可以唯一建树，关键是读到 `.` 时返回空。
- 需要一个全局下标或迭代器，不能每次从字符串开头重新读。
- 输出时不要输出 `.`，它只表示空结点。
- 中序是 `left-root-right`，后序是 `left-right-root`，顺序不要混。
- 递归构建时必须先建左子树，再建右子树，因为输入是先序。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 9: 最终可用代码（对话来源）

```
class Treenode:
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
        self.right = right

preorder = input().strip()

def build_tree(seq):
    """根据扩展二叉树的先序序列递归建树"""
    idx = 0 # 用于记录当前处理的位置
    def helper():
        nonlocal idx
        if idx >= len(seq):
            return None

        val = seq[idx]
        idx += 1

        # 遇到 '.' 表示空节点
        if val == '.':
            return None
```

```
# 创建当前节点
node = Treenode(val)

# 递归构建左子树
node.left = helper()
# 递归构建右子树
node.right = helper()

return node

return helper()

def inorder(root, ans):
    if root:
        inorder(root.left, ans)
        ans.append(root.val)
        inorder(root.right, ans)
    return ans

def postorder(root, ans):
    if root:
        postorder(root.left, ans)
        postorder(root.right, ans)
        ans.append(root.val)
    return ans

# 构建二叉树
root = build_tree(preorder)
ans1 = []
ans2 = []
ans1 = inorder(root, ans1)
ans2 = postorder(root, ans2)
print(''.join(ans1))
print(''.join(ans2))
```

复盘记忆点

- 扩展二叉树题：看到. 就返回 None。
- 先序建树顺序：root, left, right。
- 遍历输出时忽略空结点。

1.10 53: 根据二叉树中后序序列建树

题干与样例

题目编号与名称：53: 根据二叉树中后序序列建树

要求：已知中序遍历和后序遍历，结点互不相同，输出前序遍历。

输入：两行：第一行中序序列，第二行后序序列。

输出：输出一行前序遍历。

样例输入：

```
DUPR
DPRU
```

样例输出：

```
UDRP
```

错误梳理

- 后序最后一个字符是根，不是第一个。
- 在中序中找到根后，左边是左子树，右边是右子树。
- 切分后序时仍然要靠左子树长度，右子树在根之前。
- 输出前序是 $\text{root} + \text{left} + \text{right}$ 。
- 长度不超过 26，字符串递归足够。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 10: 最终可用代码（对话来源）

```
class Treenode:
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
        self.right = right

def Buildtree(postorder, inorder):
    if not postorder: # 这个判断是非常有意义的
        return
    root_val = postorder[-1] # 赋值
    root = Treenode(root_val) # 赋一个结点

    x = inorder.index(root_val)

    ino_left = inorder[:x]
    ino_right = inorder[x+1:]

    l = len(ino_left)

    post_left = postorder[0:l]
    post_right = postorder[l:-1]

    root.left = Buildtree(post_left, ino_left)
    root.right = Buildtree(post_right, ino_right)

    return root

def preorder(root, ans):
    if root:
        ans.append(root.val)
        preorder(root.left, ans)
        preorder(root.right, ans)
    return ans

io = input()
po = input()
ans = []
root = Buildtree(po, io)

ans = preorder(root, ans)

print("".join(ans))
```

复盘记忆点

- 中序 + 后序：根在后序末尾。
- 靠中序划分左右，靠左子树长度切后序。
- 输出前序： $\text{root} + \text{left} + \text{right}$ 。

1.11 54: 验证二叉搜索树

题干与样例

题目编号与名称: 54: 验证二叉搜索树

要求: 给定带 # 占位的层次遍历序列, 判断它是否能构成合法 BST。

输入: 一行若干字符串, # 表示空结点, 非空结点为唯一整数。

输出: 合法输出 YES, 否则输出 NO。

样例输入:

```
10 5 15 2 7 12 20
```

样例输出:

```
YES
```

错误梳理

- 不能只判断每个结点与左右孩子的大小关系; 必须检查整棵子树的取值范围。
- 例如右子树中的所有点都必须大于根, 不只是右孩子大于根。
- # 是占位, 不要转成整数。
- 层次数组中下标 i 的孩子是 $2i+1$ 和 $2i+2$ 。
- 边界必须严格: 左 $<$ root $<$ 右, 不能允许等号。

代码来源: 本题代码优先沿用你在对话中贴出的最终/可用版本; 如果当前对话记录中没有完整用户最终代码, 则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 11: 最终可用代码 (对话来源)

```
from collections import deque

# 定义节点
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

# 构建二叉树
def build_tree_level(level_list):
    if not level_list:
        return None
    root_val = level_list[0]
    root = Node(int(root_val))
    queue = deque([root])
    i = 1
    while queue and i < len(level_list):
        node = queue.popleft()
        # 左孩子
        if i < len(level_list):
            if level_list[i] != '#':
                node.left = Node(int(level_list[i]))
                queue.append(node.left)
            i += 1
        # 右孩子
        if i < len(level_list):
            if level_list[i] != '#':
                node.right = Node(int(level_list[i]))
                queue.append(node.right)
            i += 1
    return root
```



```
# 验证 BST
def is_bst(node, low=-float('inf'), high=float('inf')):
    if node is None:
        return True
    val = node.val
    if val <= low or val >= high:
        return False
    return is_bst(node.left, low, val) and is_bst(node.right, val, high)

# 主程序
level_input = input().split()
root = build_tree_level(level_input)

if is_bst(root):
    print("YES")
else:
    print("NO")
```

复盘记忆点

- BST 验证的本质是“范围传递”，不是只看父子。
- 左子树上界变成根值，右子树下界变成根值。
- 层次遍历带 # 时可以直接用数组下标递归。

2 图、BFS、最短路、生成树与拓扑

2.1 1: 单词序列

题干与样例

题目编号与名称：1: 单词序列

要求：给出开始单词、结束单词和词典，每次只能改一个字母，中间单词必须在词典中，求最短转换序列长度。

输入：第一行开始单词和结束单词；第二行词典单词，空格分隔。

输出：输出最短转换序列长度；无法转换输出 0。

样例输入：

```
hit cog
hot dot dog lot log
```

样例输出：

```
5
```

错误梳理

- 这是无权最短路，应该用 BFS，不是 DFS。
- 开始单词和结束单词可以不在词典中，所以要把它加入候选点集合。
- 只差一个字母才能连边，不能用字符串相似度或排序判断。
- 距离按序列长度算，hit->...->cog 包含起点和终点，所以起点距离为 1。
- visited 要在入队时标记，避免同一层反复入队。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 12: 最终可用代码（对话来源）

```

from collections import deque

class Vertex:
    def __init__(self, key):
        self.id = key
        self.connectedTo = {}
        self.color = 'white' # 顶点访问状态 (white=未访问, gray=正在访问, black=已完成)
        self.distance = float('inf') # 当前顶点到起始点的距离, 初始设为无穷大
        self.pred = None # 处理这个顶点的前驱, 用于记录路径

    def addNeighbor(self, nbr, weight=0):
        self.connectedTo[nbr] = weight # 是字典, nbr是顶点对象的key

    def __str__(self):
        return str(self.id) + ' connectedTo: ' + str([x.id for x in self.connectedTo])

    def getConnections(self):
        return self.connectedTo.keys()

    def getId(self):
        return self.id

    def getWeight(self, nbr):
        return self.connectedTo[nbr]

    def getPred(self):
        return self.pred

    def setPred(self, pred):
        self.pred = pred

    def getColor(self):
        return self.color

    def setColor(self, color):
        self.color = color

    def getDistance(self):
        return self.distance

    def setDistance(self, distance):
        self.distance = distance

class Graph:
    def __init__(self):
        self.vertList = {}
        self.numVertices = 0

    def addVertex(self, key):
        self.numVertices = self.numVertices + 1
        newVertex = Vertex(key)
        self.vertList[key] = newVertex
        return newVertex

    def getVertex(self, n): # 通过key查找顶点
        if n in self.vertList:
            return self.vertList[n]
        else:
            return None

    def __contains__(self, n):
        return n in self.vertList

    def addEdge(self, f, t, cost=0): # f起点, t终点
        if f not in self.vertList:
            nv = self.addVertex(f)

        if t not in self.vertList:

```

```

        nv = self.addVertex(t)
        self.vertList[f].addNeighbor(self.vertList[t], cost)

def getVertices(self):
    return self.vertList.keys()

def __iter__(self):
    return iter(self.vertList.values())

def buildGraph(wordlist):
    d = {}
    g = Graph()

    # 第一步: 建立 bucket
    # bucket 的作用: 把“只差一个字母”的单词放到同一组里
    for word in wordlist:

        # 依次把单词的每一个位置替换成 "_"
        # 例如 fool:
        # _ool, f_ol, fo_l, foo_
        for i in range(len(word)):
            bucket = word[:i] + '_' + word[i + 1:]

            # 如果这个 bucket 已经存在, 就把当前单词加入这个 bucket
            if bucket in d:
                d[bucket].append(word)

            # 如果这个 bucket 不存在, 就创建一个新的列表
            else:
                d[bucket] = [word]

    # 第二步: 根据 bucket 建图
    # 同一个 bucket 中的单词, 说明它们只差一个字母
    # 因此这些单词之间应该有边
    for bucket in d.keys():
        for word1 in d[bucket]:
            for word2 in d[bucket]:

                # 避免自己和自己连边
                if word1 != word2:
                    g.addEdge(word1, word2)

    # 返回构建好的图
    return g

def bfs(g, start):
    # 将起始顶点到自己的距离设置为 0
    start.setDistance(0)

    # 起始顶点没有前驱节点
    start.setPred(None)

    start.setColor('gray')

    # 创建一个队列, 用于 BFS 的“先进先出”遍历
    vertQueue = deque()

    # 先把起始顶点放入队列
    vertQueue.append(start)

    # 只要队列不为空, 就继续搜索
    while len(vertQueue) > 0:

        # 取出队首顶点, 作为当前正在访问的顶点
        currentVert = vertQueue.popleft()

        # 遍历当前顶点的所有邻接点
        for nbr in currentVert.getConnections():

            # 如果邻接点还是 white, 说明它还没有被访问过
            if nbr.getColor() == 'white':

```

```

# 将邻接点标记为 gray，表示已经发现，但还没有完全处理完
nbr.setColor('gray')

# 邻接点距离 = 当前顶点距离 + 1
# 因为 BFS 每走一条边，距离增加 1
nbr.setDistance(currentVert.getDistance() + 1)

# 记录邻接点的前驱节点
# 说明 nbr 是从 currentVert 访问到的
nbr.setPred(currentVert)

# 把这个邻接点加入队列，等待之后继续访问它的邻居
vertQueue.append(nbr)

# 当前顶点的所有邻居都检查完后，将其标记为 black
# black 表示这个顶点已经彻底处理完成
currentVert.setColor('black')

start, end = input().split()
wordlist = input().split()

wordlist.append(start)
wordlist.append(end)

wordlist = list(set(wordlist))

g = buildGraph(wordlist)

s = g.getVertex(start)
e = g.getVertex(end)

if s is None or e is None:
    print(0)
else:
    bfs(g, s)

    if e.getDistance() == float('inf'):
        print(0)
    else:
        print(e.getDistance() + 1)

```

复盘记忆点

- “最短转换次数/序列长度”通常用 BFS。
- 起点到起点的序列长度是 1。
- 单词数不超过 30， $O(n^2 * L)$ 建图或直接枚举都能过。

2.2 2: 仙岛求药

题干与样例

题目编号与名称：2: 仙岛求药

要求：在 $M \times N$ 迷宫中从 @ 走到 *，# 不可走，. 可走，求最少经过方格数；包括起点方格。多组数据，0 0 结束。

输入：每组先读 M,N；接下来 M 行地图。

输出：每组输出最少方格数；不可达输出 -1。

样例输入：

```

8 8
.@###...#
#...#.#
#.#.##..

```

```

..#..###.
#.#...#.
..###.#.
...#.*..
.#...###
0 0

```

样例输出：

```
10
```

错误梳理

- visited 的维度必须是 M 行 N 列，不是 N*N。
- 题目要求计数包括初始位置，所以起点步数/距离应从 1 开始。
- 多组数据直到 0 0，不能只读一张地图。
- 找到 @ 后要记录起点；地图中 * 是终点。
- 越界判断用 $nx < 0$ or $nx \geq M$ or $ny < 0$ or $ny \geq N$ ，行列不能写反。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 13: 最终可用代码（对话来源）

```

import collections

def bfs(maze, M, N, sx, sy):

    # 创建队列，相当于 BFS 中的 Open 表
    q = collections.deque()

    # visited[i] 表示位置 i 是否已经访问过
    # 用于判重，避免反复访问同一个位置
    visited = [[False] * N for i in range(M)]

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    q.append((sx, sy, 0))

    visited[sx][sy] = True

    # 只要队列不为空，就继续 BFS
    while len(q) > 0:
        # 取出队首元素
        x, y, dis = q.popleft()

        # 如果当前位置已经是目标位置 K，输出步数并结束
        if maze[x][y] == '*':
            return dis

        for dx, dy in directions:
            nx = x + dx
            ny = y + dy

            if nx < 0 or nx >= M or ny < 0 or ny >= N:
                continue

            if visited[nx][ny] == True:
                continue

            if maze[nx][ny] == '#':
                continue

            visited[nx][ny] = True

```

```

        q.append((nx,ny,dis+1))
    return -1

while True:
    M, N = map(int, input().split())

    if M == 0 and N == 0:
        break

    maze = []
    sx, sy = -1, -1

    for i in range(M):
        row = input().strip()
        maze.append(row)

        for j in range(N):
            if row[j] == '@':
                sx, sy = i, j

    print(bfs(maze, M, N, sx, sy))

```

复盘记忆点

- 网格 BFS 三件套：队列、visited、四方向。
- 先判越界，再判障碍，再判 visited。
- 题目说“经过方格数包括起点”时，起点距离设为 1。

2.3 3: 变换的迷宫

题干与样例

题目编号与名称：3: 变换的迷宫

要求：从 S 到 E，每步耗时 1，不能停留；# 在当前时间为 K 的倍数时可通过，其余时间不可通过，求最短时间。

输入：第一行 T；每组 R,C,K；接下来 R 行迷宫。

输出：可达输出最短时间，不可达输出 Oop!。

样例输入：

```

1
6 6 2
...S..
...#..
.#....
...#..
...#..
..#E#.

```

样例输出：

```

7

```

错误梳理

- 状态不能只用 (x,y)，因为同一格在不同 $\text{time} \% K$ 下后续可走性不同。
- visited 应为 $R * C * K$ 三维，第三维是时间模 K。
- 移动到下一格后的时间是 $t+1$ ，判断 # 是否可走要看 $\text{next_time} \% K$ 。
- 题目说不能停留，所以不能加入原地不动方向。

- 输出失败字符串是 Oop!, 大小写和感叹号都要准确。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 14: 最终可用代码（对话来源）

```
import collections

def bfs(maze, M, N, sx, sy, K):

    # 创建队列，相当于 BFS 中的 Open 表
    q = collections.deque()

    # visited[i] 表示位置 i 是否已经访问过
    # 用于判重，避免反复访问同一个位置
    visited = [[False] * K for i in range(N)] for j in range(M)]

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    q.append((sx, sy, 0))

    visited[sx][sy][0] = True

    # 只要队列不为空，就继续 BFS
    while len(q) > 0:
        # 取出队首元素
        x, y, dis = q.popleft()

        # 如果当前位置已经是目标位置 K，输出步数并结束
        if maze[x][y] == 'E':
            return dis

        for dx, dy in directions:
            nx = x + dx
            ny = y + dy

            if nx < 0 or nx >= M or ny < 0 or ny >= N:
                continue

            if visited[nx][ny][(dis+1)%K] == True:
                continue

            if maze[nx][ny] == '#' and (dis+1)%K != 0:
                continue

            visited[nx][ny][(dis+1)%K] = True

            q.append((nx, ny, dis+1))

    return -1

n = int(input())

while n:
    M, N, K = map(int, input().split())
    maze = []
    for i in range(M):
        row = input().strip()
        maze.append(row)

        for j in range(N):
            if row[j] == 'S':
                sx, sy = i, j

    ans = bfs(maze, M, N, sx, sy, K)
    if ans == -1:
        print("Oop!")
    else:
        print(ans)
    n -= 1
```

复盘记忆点

- 动态迷宫 BFS：位置 + 时间模数一起作为状态。
- 判断石头能不能走，要用“到达那一刻”的时间。
- 不能停留，就只有四个移动方向。

2.4 1: 拓扑排序

题干与样例

题目编号与名称：1: 拓扑排序

要求：给出有向图，输出拓扑排序序列；在同等条件下编号小的顶点在前。

输入：第一行 v, a ；接下来 a 行每行一条弧的两个顶点编号。

输出：输出若干个用空格隔开的顶点，格式为 $v1\ v3\ \dots$ 。

样例输入：

```
6 8
1 2
1 3
1 4
3 2
3 5
4 5
6 4
6 5
```

样例输出：

```
v1 v3 v2 v6 v4 v5
```

错误梳理

- “编号小的顶点在前”不等于每次随使用普通队列；如果有多个入度为 0，要用最小堆。
- 边是有方向的，读入 $a\ b$ 表示 $a \rightarrow b$ ，入度加在 b 上。
- 输出要带字母 v 前缀，不是只输出数字。
- 如果用 `queue.Queue`，无法保证同层编号最小优先。
- 邻接表下标要统一，顶点通常从 1 到 v 。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 15: 最终可用代码（对话来源）

这道题的重点是因为他需要在距离一致的时候用较小的序号插入，所以需要堆排序

```
import heapq
```

```
def topoSort(G, n):
    # inDegree[i] 表示顶点 i 的入度
    # 这里顶点编号从 1 到 n，所以数组开 n + 1
    inDegree = [0] * (n + 1)

    # 统计每个顶点的入度
    for u in range(1, n + 1):
        for v in G[u]:
            inDegree[v] += 1
```



```

# 小根堆，用来存当前所有入度为 0 的顶点
# heapq 每次会弹出编号最小的顶点
heap = []

# 把一开始所有入度为 0 的顶点加入堆
for i in range(1, n + 1):
    if inDegree[i] == 0:
        heapq.heappush(heap, i)

# 保存拓扑排序结果
seq = []

# 只要堆不为空，就继续取点
while heap:
    # 取出当前所有入度为 0 的点中编号最小的那个
    u = heapq.heappop(heap)

    # 加入拓扑序列
    seq.append(u)

    # 删除 u 指向的所有边
    for v in G[u]:
        inDegree[v] -= 1

        # 如果 v 的入度变成 0，说明它可以被选择了
        if inDegree[v] == 0:
            heapq.heappush(heap, v)

# 如果没有输出所有顶点，说明图中有环
if len(seq) != n:
    return None

return seq

# 读入顶点数和弧数
n, a = map(int, input().split())

# G[u] 存放所有从 u 出发的点
# 顶点编号从 1 到 n
G = [[] for _ in range(n + 1)]

# 读入 a 条有向边
for _ in range(a):
    u, v = map(int, input().split())
    G[u].append(v)

# 求拓扑排序
ans = topoSort(G, n)

# 按题目要求输出 v1 v3 v2 ...
if ans is not None:
    print(" ".join("v" + str(x) for x in ans))

```

复盘记忆点

- 拓扑排序核心：入度数组 + 入度为 0 的集合。
- 要求编号小优先时，把队列换成最小堆。
- 输出格式常常比算法更容易扣分。

2.5 2: 丛林中的路

题干与样例

题目编号与名称：2: 丛林中的路

要求：给定若干村庄之间道路维护费用，要求保留一些道路使所有村庄连通且总费用最小，即求最小生成树总权值。

输入：多组数据；每组第一行 n ，随后 $n-1$ 行按字母给边；0 结束。

输出：每组输出最小维护费用。

样例输入：

```
9
A 2 B 12 I 25
B 3 C 10 H 40 I 8
C 2 D 18 G 55
D 1 E 44
E 2 F 60 G 38
F 0
G 1 H 35
H 1 I 35
3
A 2 B 10 C 40
B 1 C 20
0
```

样例输出：

```
216
30
```

错误梳理

- 这是 MST，不是最短路；目标是让全图连通且总边权最小。
- 输入每行是变长格式：字母、k、再跟 k 组“字母权值”。
- 村庄用 A,B,C... 编号，需要用 `ord(ch)-ord("A")` 转成下标。
- Kruskal 要对所有边按权值排序，并查集合并。
- 不要把无向边重复加两次影响不大，但本题输入本来只给后继村庄，按一条边处理即可。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 16: 最终可用代码（对话来源）

```
import heapq
import sys

class Graph_adj_mat:

    def __init__(self, num_vertices):
        self.numVertices = num_vertices
        self.adj_matrix = [[0] * num_vertices for _ in range(num_vertices)]

    def addEdge(self, v1, v2, weight):
        self.adj_matrix[v1][v2] = weight
        self.adj_matrix[v2][v1] = weight  # 无向图

    def get_neighbors(self, idx):
        neighbors = []
        for j in range(self.numVertices):
            if self.adj_matrix[idx][j] != 0:
                neighbors.append(j)
```

```

        return neighbors

    def getWeight(self, v1, v2):
        return self.adj_matrix[v1][v2]

def prim(G, start):
    # pq 中存的是: 边权、顶点编号
    pq = []

    # visited[i] 表示顶点 i 是否已经加入最小生成树
    visited = [False] * G.numVertices

    # 从起点 start 开始, 代价为 0
    heapq.heappush(pq, (0, start))

    totalCost = 0

    while pq:
        # 取出当前边权最小的顶点
        currentCost, currentVert = heapq.heappop(pq)

        # 如果这个点已经加入生成树, 就跳过
        if visited[currentVert]:
            continue

        # 把当前点加入最小生成树
        visited[currentVert] = True

        # 加上连接这个点的边权
        totalCost += currentCost

        # 遍历 currentVert 的所有邻居
        for nextVert in G.get_neighbors(currentVert):
            if not visited[nextVert]:
                newCost = G.getWeight(currentVert, nextVert)
                heapq.heappush(pq, (newCost, nextVert))

    return totalCost

while True:
    n = int(input())
    if n == 0:
        break

    G = Graph_adj_mat(n)

    for _ in range(n - 1):
        data = input().split()

        start = data[0]
        k = int(data[1])

        v1 = ord(start) - ord('A')

        index = 2
        for i in range(k):
            end = data[index]
            weight = int(data[index + 1])

            v2 = ord(end) - ord('A')

            G.addEdge(v1, v2, weight)

            index += 2

    m = prim(G, 0)
    print(m)

```

复盘记忆点

- “连通且费用最小”基本就是最小生成树。
- Kruskal 模板：排序边 + 并查集。
- 字母编号题：A->0, B->1。

2.6 3: A Walk Through the Forest

题干与样例

题目编号与名称：3: A Walk Through the Forest

要求：办公室为 1，家为 2。无向带权图中，Jimmy 每一步必须“更接近家”：从 A 到 B 合法当且仅当 $\text{dist}(B, 2) < \text{dist}(A, 2)$ 。求从 1 到 2 的合法路线数。

输入：多组数据；每组第一行 N,M，之后 M 行 a b d；单独一行 0 结束。

输出：每组输出一行合法路线数。

样例输入：

```
5 6
1 3 2
1 4 2
3 4 3
1 5 12
4 2 34
5 2 24
7 8
1 3 1
1 4 1
3 7 1
7 4 1
7 5 1
6 7 1
5 2 1
6 2 1
0
```

样例输出：

```
2
4
```

错误梳理

- 最短路要从家 2 出发跑 Dijkstra，得到每个点到家的最短距离。
- 路线计数不是最短路条数，而是所有满足距离严格下降的路径数。
- 合法转移条件必须是 $\text{dist}[v] < \text{dist}[u]$ ，不能写成边权更小。
- DFS 需要 memo，否则会重复计算大量子问题。
- 多组输入第一行可能只有一个 0，要先判断长度或先读整行。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 17: 最终可用代码（对话来源）

```
# 先建一个graph

import sys
```

```
import heapq

class Vertex:

    def __init__(self, key):
        self.id = key

        # connectedTo 用来保存当前顶点的所有邻居
        # 格式:
        # {
        #     邻居顶点对象1: 边权1,
        #     邻居顶点对象2: 边权2
        # }
        self.connectedTo = {}

        self.color = 'white'
        self.distance = float('inf')
        self.pred = None

    def addNeighbor(self, nbr, weight=0):
        # 添加一条从 self 到 nbr 的边, 权重为 weight
        self.connectedTo[nbr] = weight

    def getConnections(self):
        # 返回所有邻居顶点对象
        return self.connectedTo.keys()

    def getId(self):
        return self.id

    def getWeight(self, nbr):
        # 返回 self 到 nbr 这条边的权重
        return self.connectedTo[nbr]

    def getPred(self):
        return self.pred

    def setPred(self, pred):
        self.pred = pred

    def getColor(self):
        return self.color

    def setColor(self, color):
        self.color = color

    def getDistance(self):
        return self.distance

    def setDistance(self, distance):
        self.distance = distance

class Graph:

    def __init__(self):
        # vertList 保存所有顶点
        # 格式:
        # {
        #     顶点编号: 顶点对象
        # }
        self.vertList = {}

        # 顶点数量
        self.numVertices = 0

    def addVertex(self, key):
        # 添加一个新顶点
        self.numVertices += 1
        newVertex = Vertex(key)
        self.vertList[key] = newVertex
        return newVertex
```

```
def getVertex(self, key):
    # 根据编号获取顶点对象
    if key in self.vertList:
        return self.vertList[key]
    else:
        return None

def __contains__(self, key):
    # 支持写法: if key in graph
    return key in self.vertList

def addEdge(self, f, t, cost=0):
    # 如果起点 f 不存在, 就先加入图中
    if f not in self.vertList:
        self.addVertex(f)

    # 如果终点 t 不存在, 也先加入图中
    if t not in self.vertList:
        self.addVertex(t)

    # 添加一条从 f 到 t 的边
    self.vertList[f].addNeighbor(self.vertList[t], cost)

def addUndirectedEdge(self, f, t, cost=0):
    # 无向图要加两条边
    # f -> t
    # t -> f
    self.addEdge(f, t, cost)
    self.addEdge(t, f, cost)

def getVertices(self):
    # 返回所有顶点编号
    return self.vertList.keys()

def __iter__(self):
    # 支持写法: for v in graph
    # 每次返回的是 Vertex 对象
    return iter(self.vertList.values())

def dijkstra(aGraph, start):
    # 初始化所有点
    for v in aGraph:
        v.setDistance(float('inf'))
        v.setPred(None)

    # 起点距离设为 0
    start.setDistance(0)

    # 小根堆中存放:
    # (当前距离, 顶点编号, 顶点对象)
    heap = []
    heapq.heappush(heap, (0, start.getId(), start))

    while heap:
        currentDist, _, currentVert = heapq.heappop(heap)

        # 如果这个距离不是最新的, 跳过
        if currentDist != currentVert.getDistance():
            continue

        # 遍历当前顶点的所有邻居
        for nextVert in currentVert.getConnections():

            # 新路径长度 = 起点到 currentVert 的距离 + currentVert 到 nextVert 的边权
            newDist = currentVert.getDistance() + currentVert.getWeight(nextVert)

            # 如果新路径更短, 就更新
            if newDist < nextVert.getDistance():
                nextVert.setDistance(newDist)
                nextVert.setPred(currentVert)
```

```

        heapq.heappush(heap, (newDist, nextVert.getId(), nextVert))

def dfs_count_routes(u, memo):
    if u.getId() == 2:
        return 1

    if memo[u.getId()] != -1:
        return memo[u.getId()]

    ans = 0

    for v in u.getConnections():
        if v.getDistance() < u.getDistance():
            ans += dfs_count_routes(v, memo)

    memo[u.getId()] = ans
    return ans

sys.setrecursionlimit(1000000)

while True:
    line = sys.stdin.readline().strip()

    if line == "0":
        break

    n, m = map(int, line.split())

    G = Graph()

    # 先创建 1 到 n 的所有顶点
    for i in range(1, n + 1):
        G.addVertex(i)

    for _ in range(m):
        a, b, d = map(int, sys.stdin.readline().split())
        G.addUndirectedEdge(a, b, d)

    dijkstra(G, G.getVertex(2))

    memo = [-1] * (n + 1)

    print(dfs_count_routes(G.getVertex(1), memo))

```

复盘记忆点

- 先算“到家距离”，再沿距离下降方向 DP。
- return memo[u] 表示之前算过；return ans 表示这次刚算完。
- 这题合法路线数量，不是最短路线数量。

2.7 4: 打怪救公主

题干与样例

题目编号与名称：4: 打怪救公主

要求：迷宫中从 * 出发救 +，数字格是怪物血量，进入会扣血；# 是墙。若能到达公主，输出剩余最大血量，否则 0。

输入：第一行 m,n,t；接下来 m 行地图。

输出：能救出则输出到达 + 时最大剩余血量，否则输出 0。

样例输入：

```

5 6 10
..*...

```

```

.#2###
5#..4#
.##9.#
.#+..#

```

样例输出：

```
4
```

错误梳理

- 这题不是普通 BFS 求最少步数，而是要最大化剩余血量。
- 同一个格子如果以更多血量到达，仍然值得继续扩展；visited 不能只是布尔值。
- 可以用 best[x][y] 记录到达该格子的最大剩余血量，只有更优才入队。
- 进入数字格时先扣血，血量必须仍然大于 0 才能继续。
- 起点 * 和终点 + 不扣血，. 也不扣血。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 18: 最终可用代码（对话来源）

```

from collections import deque

def max_blood_bfs(maze, M, N, sx, sy, B):
    q = deque()
    max_blood = [[-1]*N for _ in range(M)]
    q.append((sx, sy, B))
    max_blood[sx][sy] = B

    directions = [(-1,0),(1,0),(0,-1),(0,1)]

    while q:
        x, y, blood = q.popleft()

        for dx, dy in directions:
            nx, ny = x+dx, y+dy
            if nx<0 or nx>=M or ny<0 or ny>=N:
                continue
            cell = maze[nx][ny]
            if cell == '#':
                continue

            new_blood = blood
            if cell.isdigit():
                dmg = int(cell)
                new_blood -= dmg
                if new_blood <=0:
                    continue

            if new_blood > max_blood[nx][ny]:
                max_blood[nx][ny] = new_blood
                q.append((nx, ny, new_blood))

    # 找到公主位置
    for i in range(M):
        for j in range(N):
            if maze[i][j] == '+':
                return max_blood[i][j] if max_blood[i][j] != -1 else 0

M, N, B = map(int, input().split())
maze = []
sx, sy = -1, -1

```



```

for i in range(M):
    row = input().strip()
    maze.append(row)

    for j in range(N):
        if row[j] == '*':
            sx, sy = i, j

print(max_blood_bfs(maze, M, N, sx, sy, B))

```

复盘记忆点

- 看到“剩余最大血量”，visited 要存最优值，不是 True/False。
- 只有状态变好才继续入队。
- 血量减到 0 会死，因此必须 $nhp > 0$ 。

2.8 5: 兔子与樱花

题干与样例

题目编号与名称：5: 兔子与樱花

要求：给定地点、无向带权道路和若干查询，输出每个查询的最短路径，并按 A->(d)->B 的格式打印整条路线。

输入：先读 P 个地点；再读 Q 条道路；最后读 R 个查询。

输出：每个查询输出一行最短路径表示；起点等于终点时只输出地点名。

样例输入：

```

6
Ginza
Sensouji
Shinjukugyoen
Uenokouen
Yoyogikouen
Meijishinguu
6
Ginza Sensouji 80
Shinjukugyoen Sensouji 40
Ginza Uenokouen 35
Uenokouen Shinjukugyoen 85
Sensouji Meijishinguu 60
Meijishinguu Yoyogikouen 35
2
Uenokouen Yoyogikouen
Meijishinguu Meijishinguu

```

样例输出：

```

Uenokouen->(35)->Ginza->(80)->Sensouji->(60)->Meijishinguu->(35)->Yoyogikouen
Meijishinguu

```

错误梳理

- 不仅要求最短距离，还要求输出具体路径，因此需要记录前驱。
- 地点是字符串，先映射成编号，输出时再映射回名称。
- 多次查询且 $P < 30$ ，可以用 Floyd 预处理所有点对最短路。
- 路径恢复时要维护 next 矩阵，而不是只保存 dist。
- 起点等于终点时，直接输出地点名，不要输出箭头。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 19: 最终可用代码（对话来源）

```
class Vertex:
    def __init__(self, key, value):
        self.id = key
        self.name = value
        self.connectedTo = {}
        self.distance = float('inf')
        self.pred = None

    def addNeighbor(self, nbr, weight=0):
        self.connectedTo[nbr] = weight

    def getConnections(self):
        return self.connectedTo.keys()

    def getId(self):
        return self.name

    def getWeight(self, nbr):
        return self.connectedTo[nbr]

    def getDistance(self):
        return self.distance

    def setDistance(self, distance):
        self.distance = distance

    def getPred(self):
        return self.pred

    def setPred(self, pred):
        self.pred = pred

class Graph:
    def __init__(self):
        self.vertList = {}
        self.nameMap = {}
        self.numVertices = 0

    def addVertex(self, key, name):
        newVertex = Vertex(key, name)
        self.vertList[key] = newVertex
        self.nameMap[name] = newVertex
        self.numVertices += 1
        return newVertex

    def getVertexByName(self, name):
        return self.nameMap.get(name, None)

    def addEdgeByName(self, f_name, t_name, cost=0):
        f = self.getVertexByName(f_name)
        t = self.getVertexByName(t_name)
        f.addNeighbor(t, cost)

    def __iter__(self):
        return iter(self.vertList.values())

    def reset(self):
        for v in self:
            v.setDistance(float('inf'))
            v.setPred(None)

def dijkstra_no_pq(aGraph, start):
    aGraph.reset()
    # 设置起点到自己的距离为 0
    start.setDistance(0)
```

```

# 已经确定最短路径的顶点集合
visited = set()

# 只要还有未访问顶点，就继续循环
while len(visited) < aGraph.numVertices:

    # 在所有未访问顶点中，找到距离起点最小的顶点
    currentVert = None
    currentDist = float('inf')
    for v in aGraph:
        if v not in visited and v.getDistance() < currentDist:
            currentDist = v.getDistance()
            currentVert = v

    # 如果没有可访问顶点，说明剩余顶点不可达，结束循环
    if currentVert is None:
        break

    # 把当前顶点加入已访问集合
    visited.add(currentVert)

    # 遍历 currentVert 的所有邻接点
    for nextVert in currentVert.getConnections():

        # 计算通过 currentVert 到 nextVert 的新距离
        newDist = currentVert.getDistance() + currentVert.getWeight(nextVert)

        # 如果新距离更短，更新 nextVert 的最短距离
        if newDist < nextVert.getDistance():
            nextVert.setDistance(newDist)
            nextVert.setPred(currentVert)
def getPath(end):
    path = []

    current = end
    while current is not None:
        path.append(current)
        current = current.getPred()

    path.reverse()
    return path

def printPath(path):
    ans = path[0].getId()

    for i in range(1, len(path)):
        weight = path[i-1].getWeight(path[i])
        ans += "->(" + str(weight) + ")->" + path[i].getId()

    print(ans)

g = Graph()
n1 = int(input())

for i in range(n1):
    name = input().strip()
    g.addVertex(i, name)

n2 = int(input())
for _ in range(n2):
    f_name, t_name, weight = input().split()
    weight = int(weight)
    g.addEdgeByName(f_name, t_name, weight)
    g.addEdgeByName(t_name, f_name, weight)

n3 = int(input())
for _ in range(n3):
    s_name, e_name = input().split()
    s = g.getVertexByName(s_name)
    e = g.getVertexByName(e_name)
    dijkstra_no_pq(g, s)
    p = getPath(e)

```

```
printPath(p)
```

复盘记忆点

- 多源多查询且点数小：Floyd 很方便。
- 输出路径需要 next 矩阵恢复下一跳。
- 格式是点名->(边长)-> 点名，不是只输出总距离。

2.9 56: 拯救行动

题干与样例

题目编号与名称：56: 拯救行动

要求：在 $N \times M$ 牢房矩阵中，骑士 r 要到达公主 a。道路为 @，墙为 #，守卫为 x。骑士上下左右移动，每移动一格花费 1；如果进入守卫格，还需要额外花费 1 杀死守卫。求成功救到公主的最短时间；不可达输出 Impossible。

输入：第一行整数 S，表示数据组数。每组先输入 N 和 M，随后 N 行、每行 M 个字符，字符包括 @、#、x、r、a。

输出：每组输出最短时间；若不可能成功，输出 Impossible。

样例输入：

```
2
7 8
#@#####@
#@a#@@r@
#@@#x@@@
@@#@@@#@#
#@@@@##@@
@#@@@@@@
@@@@@@@@
13 40
@x@@@##xx@x#xxxx#@@#x@x@@#x#@x@x@x@x
xx##x@x#@@##xx@@@#@x@@@#xx@x#@x@x@
#@x#@x#@x#@@#@x#@x#xxxx@#@#@#@x@x@x@
@##x@x@x#xx#@@#xxxx@x@x@x@x@x@x@x@#
@xxxxx#@@x#@x@x@x@x@x@x@x@x@x@x@x@
#xx#@#@#xx#@@#@x@x@x@x@x@x@x@x@x@x@
#x@xxxx#@x@x@x@x@x@x@x@x@x@x@x@x@x@
xx#@#@#@x#@x#@x#@a#xx#@#@#@#@x@x@x@
x#@x@x#@#@#@x#@x@x@x@x@x@x@x@x@x@x@
#@x@x@x@x@x#@x#@x@x@x@x@x@x@x@x@x@
xx#@x@x@x@x#@x#@x@x@x@x@x@x@x@x@x@
#@#@x@x@x@x@x@x@x@x@x@x@x@x@x@x@x@
#x#@x@x@x@x@x@x@x@x@x@x@x@x@x@x@
```

样例输出：

```
13
7
```

错误梳理

- 这题不是普通 BFS，因为进入不同格子的代价不一样：普通道路代价 1，守卫 x 代价 2。所以如果直接用队列按层扩展，可能先到达的状态并不是最短时间。
- 正确思路应当是 Dijkstra：优先队列里存当前最短时间，每次弹出时间最小的位置。
- 不能把 Step 类对象直接放入 heapq 比较；如果堆元素是对象，需要定义 __lt__，更简单的写法是直接压入 (time, x, y) 元组。

- 距离数组 `dist[x][y]` 要记录到每个格子的最短时间。只有当新时间更短时，才更新并入堆。
- 读入地图时要顺便记录 `r` 和 `a` 的位置；多组数据时，每组都要重新初始化 `maze`、`dist`、`pq`。
- 遇到 `#` 必须跳过；`@`、`r`、`a` 的移动代价都是 1，`x` 的移动代价是 2。
- 可以在 Dijkstra 弹出公主位置时直接返回，因为这时已经保证是全局最短时间。

代码来源：本题代码优先沿用你在对话中贴出的最终/可用版本；如果当前对话记录中没有完整用户最终代码，则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 20: 最终可用代码（对话来源）

```
import heapq

n = int(input())

for _ in range(n):

    maze = []

    M, N = map(int, input().split())

    # 找起点和终点
    for i in range(M):
        row = input().strip()
        maze.append(row)
        for j in range(N):
            if maze[i][j] == 'r':
                sx, sy = i, j
            elif maze[i][j] == 'a':
                ex, ey = i, j

    # 优先队列 (steps, x, y)
    pq = []
    heapq.heappush(pq, (0, sx, sy))

    # 记录每个格子最短时间
    dist = [[float('inf')] * N for _ in range(M)]
    dist[sx][sy] = 0

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    found = False

    while pq:
        steps, x, y = heapq.heappop(pq)

        # 如果当前步数比记录的还大，说明已经有更短路径了
        if steps > dist[x][y]:
            continue

        # 到达终点
        if x == ex and y == ey:
            print(steps)
            found = True
            break

        # 扩展邻居
        for dx, dy in directions:
            nx = x + dx
            ny = y + dy

            if 0 <= nx < M and 0 <= ny < N:
                if maze[nx][ny] == '#':
                    continue

                # 走普通格子: +1, 守卫: +2
                if maze[nx][ny] == 'x':
```

```

        nt = steps + 2
    else:
        nt = steps + 1

    # 如果这条路径更短, 就加入队列
    if nt < dist[nx][ny]:
        dist[nx][ny] = nt
        heapq.heappush(pq, (nt, nx, ny))

if not found:
    print("Impossible")

```

复盘记忆点

- 网格题如果每一步代价完全相同, 用 BFS; 如果边权出现 1 和 2 这种差异, 用 Dijkstra。
- heapq 最稳的写法是堆里放元组: (距离, 行, 列)。
- Dijkstra 的过期状态判断: 弹出后如果 $t \neq \text{dist}[x][y]$, 说明它已经不是最新最优状态, 直接跳过。

2.10 57: 连接所有点的最小费用

题干与样例

题目编号与名称: 57: 连接所有点的最小费用

要求: 给定二维平面上的若干点, 连接任意两点的费用为曼哈顿距离 $|x_i - x_j| + |y_i - y_j|$ 。要求用最小总费用把所有点连通, 并且任意两点之间有且仅有一条简单路径, 即求完全图上的最小生成树总权重。

输入: 一行, 一个 points 数组, 例如 `[[0,0],[2,2],[3,10],[5,2],[7,0]]`。可用 `ast.literal_eval` 读成二维列表。

输出: 一个整数, 表示连接所有点的最小总费用。

样例输入:

```
[[0,0],[2,2],[3,10],[5,2],[7,0]]
```

样例输出:

```
20
```

错误梳理

- 题目说“所有点连通, 且任意两点之间有且仅有一条简单路径”, 本质就是最小生成树 MST, 不是最短路。
- 每两个点之间都可以连边, 所以这是一个完全图; 没有必要真的把所有边都存成邻接表再排序, 直接用 Prim 更省空间。
- 曼哈顿距离要写成 `abs(x1-x2)+abs(y1-y2)`, 不要误用欧氏距离。
- 输入不是普通的数字矩阵, 而是一整行 Python 风格列表; 要用 `ast.literal_eval`, 不能直接 `split()`。
- Prim 中 `minDist[i]` 表示“当前已选集合到点 i 的最小连接费用”, 每次选未加入集合且 `minDist` 最小的点。
- 每加入一个新点后, 要用它去更新所有未加入点的 `minDist`。
- 若点数较小, $O(n^2)$ Prim 比建 $O(n^2)$ 条边再 Kruskal 更直接, 也避免内存浪费。

代码来源: 本题代码优先沿用你在对话中贴出的最终/可用版本; 如果当前对话记录中没有完整用户最终代码, 则采用此前对话中我给出的可用版本。本次校订不重新设计新的算法实现。

Listing 21: 最终可用代码 (对话来源)

```
import ast
```

```
import heapq
import sys

class Vertex:
    def __init__(self, key):
        self.id = key
        self.connectedTo = {}
        self.color = 'white' # 顶点访问状态 (white=未访问, gray=正在访问, black=已完成)
        self.distance = float('inf') # 当前顶点到起始点的距离, 初始设为无穷大
        self.pred = None # 处理这个顶点的前驱, 用于记录路径

    def addNeighbor(self, nbr, weight=0):
        self.connectedTo[nbr] = weight # 是字典, nbr是顶点对象的key

    def __str__(self):
        return str(self.id) + ' connectedTo: ' + str([x.id for x in self.connectedTo])

    def getConnections(self):
        return self.connectedTo.keys()

    def getId(self):
        return self.id

    def getWeight(self, nbr):
        return self.connectedTo[nbr]

    def setDistance(self, dist):
        self.distance = dist

    def getDistance(self):
        return self.distance

    def setPred(self, pred):
        self.pred = pred

class Graph:
    def __init__(self):
        self.vertList = {}
        self.numVertices = 0

    def addVertex(self, key):
        self.numVertices = self.numVertices + 1
        newVertex = Vertex(key)
        self.vertList[key] = newVertex
        return newVertex

    def getVertex(self, n): # 通过key查找顶点
        if n in self.vertList:
            return self.vertList[n]
        else:
            return None

    def __contains__(self, n):
        return n in self.vertList

    def addEdge(self, f, t, cost=0): # f起点, t终点
        if f not in self.vertList:
            nv = self.addVertex(f)

        if t not in self.vertList:
            nv = self.addVertex(t)
        self.vertList[f].addNeighbor(self.vertList[t], cost)

    def getVertices(self):
        return self.vertList.keys()

    def __iter__(self):
        return iter(self.vertList.values())
```

```

def prim(G, start):
    # heapq 实现的优先队列
    pq = []

    # 记录哪些顶点已经加入最小生成树
    inTree = set()

    # 初始化所有顶点
    for v in G:
        v.setDistance(sys.maxsize)
        v.setPred(None)

    # 起点距离设为 0
    start.setDistance(0)

    # 修改: heapq 里面放三元组
    # (当前最小边权, 顶点编号, 顶点对象)
    # 加顶点编号是为了防止两个 Vertex 对象不能比较而 Runtime Error
    heapq.heappush(pq, (0, start.getId(), start))

    totalCost = 0

    while pq:
        # 相当于 pq.delMin()
        currentDistance, currentId, currentVert = heapq.heappop(pq)

        # 修改: 如果这个点已经加入生成树, 就跳过
        # 因为 heapq 没有 decreaseKey, 旧的较大距离可能还留在堆里
        if currentVert in inTree:
            continue

        # 当前顶点正式加入最小生成树
        inTree.add(currentVert)

        # 当前边权加入答案
        totalCost += currentDistance

        # 遍历邻接点
        for nextVert in currentVert.getConnections():

            newCost = currentVert.getWeight(nextVert)

            # 修改: 原来是 if nextVert in pq
            # heapq 不能直接判断某个点是否还在队列中
            # 所以改成判断 nextVert 是否还没加入生成树
            if nextVert not in inTree and newCost < nextVert.getDistance():

                nextVert.setPred(currentVert)

                nextVert.setDistance(newCost)

            # 修改: heapq 没有 decreaseKey
            # 所以直接把新的更小距离重新压入堆
            heapq.heappush(pq, (newCost, nextVert.getId(), nextVert))

    return totalCost

input_str = input().strip()
points = ast.literal_eval(input_str)

g = Graph()

for i in range(len(points)):
    for j in range(i, len(points)):
        w = abs(points[i][0] - points[j][0]) + abs(points[i][1] - points[j][1])
        g.addEdge(i, j, w)
        g.addEdge(j, i, w)

result = prim(g, g.getVertex(0))

```



```
print(result)
```

复盘记忆点

- 看到“把所有点/村庄/节点连接起来，总代价最小”就优先想到 MST。
- 点之间天然都有边、边权可即时计算时，Prim 常比 Kruskal 更方便。
- 这题和“丛林中的路”同属最小生成树：前者是坐标完全图，后者是显式给边图。

3 全局输入与调试清单

检查项	考前复盘说明
读题先看输出口径	最短步数、经过格子数、序列长度、剩余血量、路线数量是不同目标。仙岛求药计数包括起点，所以从 1 开始；变换迷宫输出时间，所以从 0 开始。
多组数据结束条件	常见结束方式有 EOF、0、0 0、先给 T。A Walk Through the Forest 是单独一行 0 结束；二叉树编号子树是 0 0 结束；前中序建树是 EOF。
行列顺序	网格题统一使用 maze[row][col]，也就是 x 表示行、y 表示列。visited 的大小必须是 [行数][列数]，不要写成 [N][N]。
visited 是布尔还是最优值	普通最短路 BFS 用 bool visited；动态迷宫要 visited[x][y][t%K]；打怪救公主要 best[x][y] 存最大剩余血量。
何时标记 visited	BFS 通常在入队时标记，避免同一个状态被重复加入队列。若状态可能被更优值更新，则不能用布尔 visited。
边界判断顺序	先判断越界，再判断墙/障碍，再判断是否访问过。否则容易数组越界 RE。
起点距离初始化	题目说“经过的方格数包括初始位置”，起点距离为 1；题目说“花费时间”，起点时间为 0。
图是有向还是无向	拓扑排序是有向图；丛林中的路、A Walk Through the Forest、兔子与樱花都是无向图。无向图读边时两边都要加。
最短路还是最小生成树	“从 A 到 B 最短”是最短路；“保证所有点连通且总费用最小”是最小生成树。不要把 Prim/Kruskal 和 Dijkstra 混用。
编号从 0 还是 1 开始	题目顶点编号若为 1..n，就开 n+1 数组；字母编号 A..Z 用 ord(ch)-ord('A')。
输出格式	是否有前缀 v、是否有箭头、是否有空格、是否末尾无空格，都要按样例。拓扑排序输出 v1 v3；兔子与樱花输出 A->(d)->B。
递归出口	树题的空结点通常返回空串或 0；扩展二叉树遇到. 返回 None；编号为 -1 的孩子深度为 0。
树的高度口径	按结点数：空树 0、叶子 1。按边数：根高度 0。题目 49 按结点数；题目 27 转换高度按样例可知是边数口径。
字符串建树切分	前序 + 中序：根是 pre[0]；中序 + 后序：根是 post[-1]。切分另一个序列时一定用左子树长度。
heapq 常见拼写	import heapq; heapq.heapify(h); heapq.heappush(h,x); heapq.heappop(h)。不要写 headq 或 heapop。
Dijkstra 过期节点	弹出 (d,u) 后若 d != dist[u]，说明是旧状态，要 continue。
DFS + 记忆化	A Walk Through the Forest 的路线计数必须 memo。return memo[u] 是复用旧答案；return ans 是本次刚算完的结果。

4 考前模板索引

题面关键词	模板	关键判断
最少步数、最少方格数	普通 BFS	队列里放 (x,y,dist)，visited 二维；若包括起点，dist 从 1 开始。
迷宫随时间变化	状态 BFS	状态必须包含时间模数，例如 (x,y,t%K)。
同一位置可用更好资源重访	最优值 BFS/SPFA 思想	visited 改成 best 数组，只有更优状态才入队。
每次只能改变一个字母	单词图 BFS	单词作为点，只差一个字母连边，起点距离为序列长度 1。
同等条件编号小优先	堆优化拓扑	入度为 0 的点放最小堆，不用普通队列。
所有点连通且费用最小	MST	Kruskal：边排序 + 并查集；Prim：点扩展 + 堆。
从某点到所有点最短	Dijkstra	非负权图；dist 初始化 INF，起点为 0，堆中弹最小。
多源多查询最短路	Floyd	点数小，用三重循环，并用 next 矩阵恢复路径。
合法路线数量	最短路 + DP	先算评价函数 dist，再按 dist 严格下降方向 DFS 记忆化。
前序 + 中序建树	字符串递归	root=pre[0]；中序划分；输出后序 left+right+root。
中序 + 后序建树	字符串递归	root=post[-1]；中序划分；输出前序 root+left+right。
扩展二叉树先序	递归读入	遇到. 返回空；先建左再建右。
括号嵌套树	括号深度解析	根是第一个字符；只按最外层逗号切左右子树。
BST 插入和层序	BST + deque	重复值忽略；层序用 deque.popleft()。
验证 BST	范围递归	每个结点携带 (low, high)，不能只看父子大小。
Huffman/WPL	最小堆	每次取两个最小值合并，答案加合并和。
完全二叉树编号	区间扩展	子树每一层是连续编号区间 [left,right]。

最小可背 BFS 模板

```
from collections import deque
q = deque([(sx, sy, 0)])
visited = [[False] * C for _ in range(R)]
visited[sx][sy] = True
while q:
    x, y, d = q.popleft()
    for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
        nx, ny = x + dx, y + dy
        if nx < 0 or nx >= R or ny < 0 or ny >= C:
            continue
        if visited[nx][ny] or grid[nx][ny] == '#':
            continue
        visited[nx][ny] = True
        q.append((nx, ny, d + 1))
```

最小可背 Dijkstra 模板

```
import heapq
INF = 10**18
dist = [INF] * (n + 1)
dist[start] = 0
pq = [(0, start)]
while pq:
    d, u = heapq.heappop(pq)
    if d != dist[u]:
        continue
    for v, w in graph[u]:
        nd = d + w
        if nd < dist[v]:
            dist[v] = nd
            heapq.heappush(pq, (nd, v))
```

最小可背并查集模板

```
parent = list(range(n))

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False
    parent[rb] = ra
    return True
```